Command

Season: 3 · Behavioral · Category: GoF · Level: Intermediate · Build with Waqar

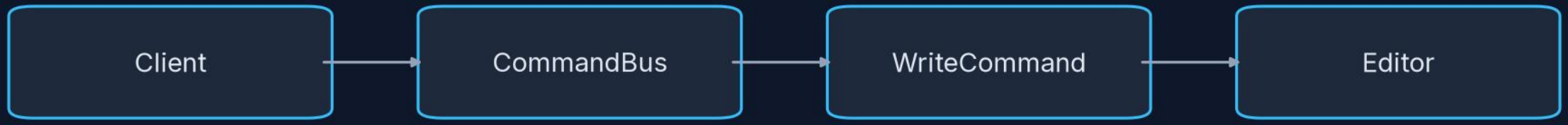
Today's Hook

- Today: Command as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Command structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Command
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/gof/behavioral/command/CommandBus.java
package com.buildwithwaqar.patterns.gof.behavioral.command;

import java.util.ArrayDeque;
import java.util.Deque;

interface Command { void execute(); void undo(); }

final class Editor {
    private final StringBuilder text = new StringBuilder();
    void append(String s) { text.append(s); }
    void deleteLast(int n) { text.delete(Math.max(0, text.length() - n), text.length()); }
    String text() { return text.toString(); }
}

record WriteCommand(Editor editor, String chunk) implements Command {
    public void execute() { editor.append(chunk); }
    public void undo() { editor.deleteLast(chunk.length()); }
}

/** Invoker with undo stack. */
public final class CommandBus {
    private final Deque<Command> history = new ArrayDeque<>();
```

Implementation (2)

```
    public void run(Command command) {
        command.execute();
        history.push(command);
    }

    public void undo() {
        if (!history.isEmpty()) history.pop().undo();
    }
}
```

Demo.main

```
package com.buildwithwaqar.patterns.gof.behavioral.command;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Command walkthrough ===");
    var editor = new Editor();
    var bus = new CommandBus();
    bus.run(new WriteCommand(editor, "Build "));
    bus.run(new WriteCommand(editor, "with "));
    bus.run(new WriteCommand(editor, "Waqar"));
    System.out.println("text=" + editor.text());
    bus.undo();
    System.out.println("afterUndo=" + editor.text());
    bus.undo();
    System.out.println("afterSecondUndo=" + editor.text());
  }
}
```

Walkthrough Output

```
=== Command walkthrough ===
text=Build with Waqar
afterUndo=Build with
afterSecondUndo=Build
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Command
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Command → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.